



Module 10

Conservative Force and Potential Energy – General Formulation

- Notations
- Discrete Equations of Motion

Goal



- Become familiar with notations and conventions
- Formulate and solve discrete equations of motion



- Read “*Chapter 4: Conservative force and potential energy*” of Course Notes



When only conservative forces are involved

$$m_i \ddot{q}_i + \frac{\partial E_{\text{potential}}}{\partial q_i} = 0$$

$$\mathbf{M} \ddot{\mathbf{q}} + \frac{\partial E_{\text{potential}}}{\partial \mathbf{q}} = \mathbf{0}$$

Mass matrix ($N = 3$):

$$\mathbf{M} = \begin{bmatrix} m_1 & 0 & 0 \\ 0 & m_2 & 0 \\ 0 & 0 & m_3 \end{bmatrix} = m_i \delta_{ij}$$

DOF Vector ($N = 3$):

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$$



Conservative forces and Viscous Forces

$$m_i \ddot{q}_i + \frac{\partial E_{\text{potential}}}{\partial q_i} + \boxed{c_i \dot{q}_i} = 0$$



Damping force: $-c_i \dot{q}_i$, where c_i (unit: N-s/m) is related to viscous damping.

$$\mathbf{M} \ddot{\mathbf{q}} + \frac{\partial E_{\text{potential}}}{\partial \mathbf{q}} + \mathbf{C} \dot{\mathbf{q}} = \mathbf{0}$$

Damping matrix ($N = 3$):

$$\mathbf{C} = \begin{bmatrix} c_1 & 0 & 0 \\ 0 & c_2 & 0 \\ 0 & 0 & c_3 \end{bmatrix} = c_i \delta_{ij}$$



Discrete (in time) equations of motion

$$m_i \ddot{q}_i + \frac{\partial E_{\text{potential}}}{\partial q_i} + c_i \dot{q}_i = 0$$

$$f_i \equiv \frac{m_i}{\Delta t} \left[\frac{q_i(t_{k+1}) - q_i(t_k)}{\Delta t} - \dot{q}_i(t_k) \right] + \frac{\partial E_{\text{potential}}}{\partial q_i} + c_i \frac{q_i(t_{k+1}) - q_i(t_k)}{\Delta t} = 0$$

$$\mathbb{J}_{ij} = \frac{\partial f_i}{\partial q_j} = \mathbb{J}_{ij}^{\text{inertia}} + \mathbb{J}_{ij}^{\text{potential}} + \mathbb{J}_{ij}^{\text{viscous}},$$

where

$$\mathbb{J}_{ij}^{\text{inertia}} = \frac{m_i}{\Delta t^2} \delta_{ij},$$

$$\mathbb{J}_{ij}^{\text{potential}} = \frac{\partial^2 E_{\text{potential}}}{\partial q_i \partial q_j},$$

$$\mathbb{J}_{ij}^{\text{viscous}} = \frac{c_i}{\Delta t} \delta_{ij}.$$



Module 11

Example Problem: Rigid Spheres Connected by Springs in Viscous Flow

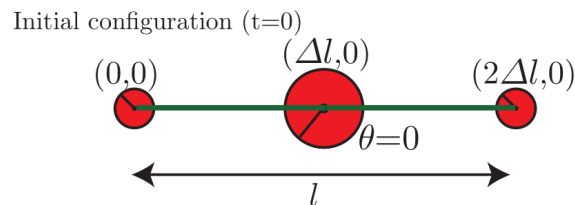
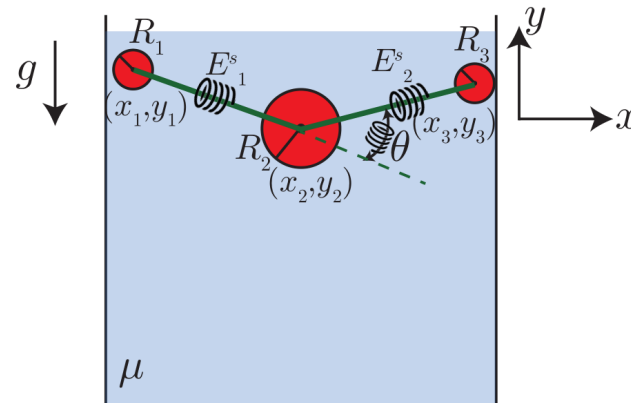
- Equations of Motion
- Advice on Programming Implementation

Goal



- Setup a simple problem with three rigid spheres connected by springs in a viscous flow
- Formulate equations of motion

- Read “*Chapter 4: Conservative force and potential energy*” of Course Notes



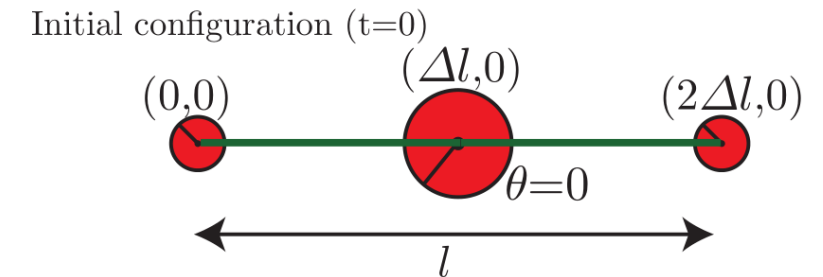
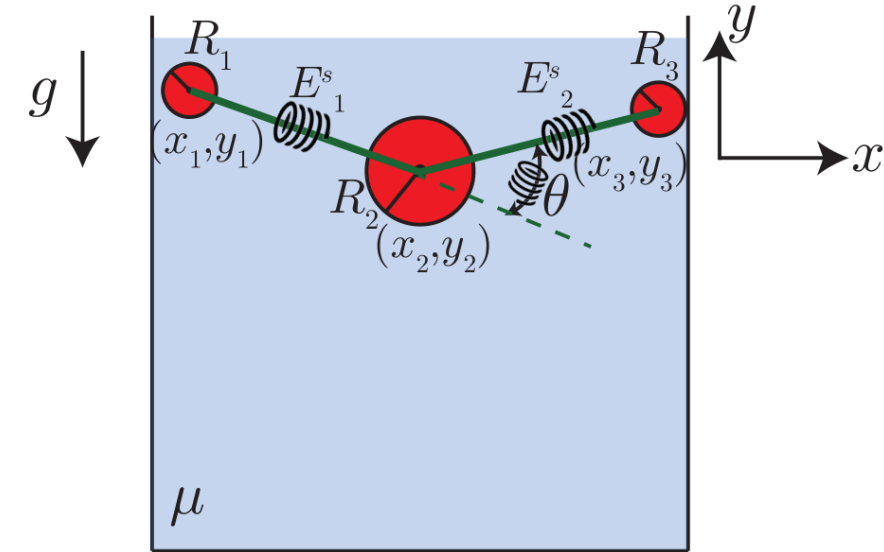


Problem Setup

Three spheres are connected with two linear springs and one torsional spring.

- Radii of the sphere (R_1, R_2, R_3) are known
- Viscosity of the fluid (μ) is known
- Initial configuration (Δl) is known
- Densities of the spheres (ρ_{metal}) and fluid (ρ_{fluid}) are known

Problem: simulate the shape of the system as a function of time



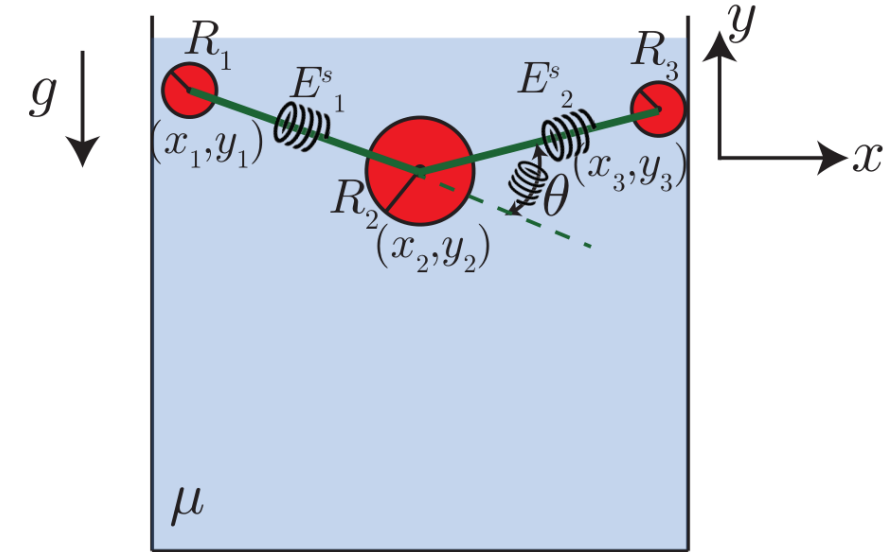


Degrees of Freedom

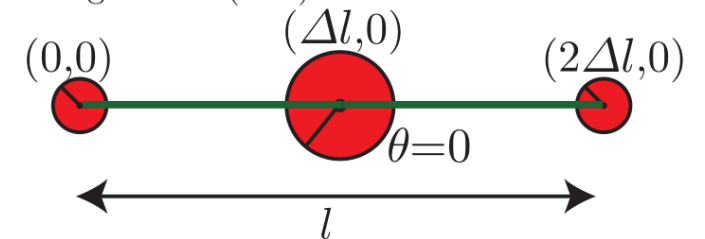
DOF Vector ($N = 6$):

$$\mathbf{q} = \begin{bmatrix} x_1 \\ y_1 \\ x_2 \\ y_2 \\ x_3 \\ y_3 \end{bmatrix}$$

Keep in mind: θ can be determined from $\mathbf{q}$
and we do not need θ as an additional DOF



Initial configuration ($t=0$)





Continuous Equations of Motion

$$m_i \ddot{q}_i + \frac{\partial E^{\text{elastic}}}{\partial q_i} + c_i \dot{q}_i - \boxed{w_i} = 0$$



Weight (unit: N) associated with the i -th DOF.

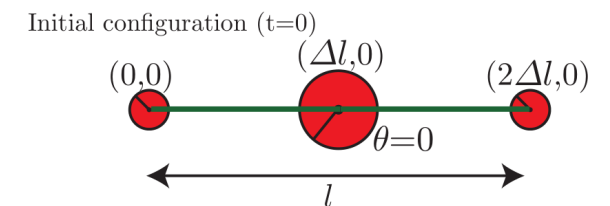
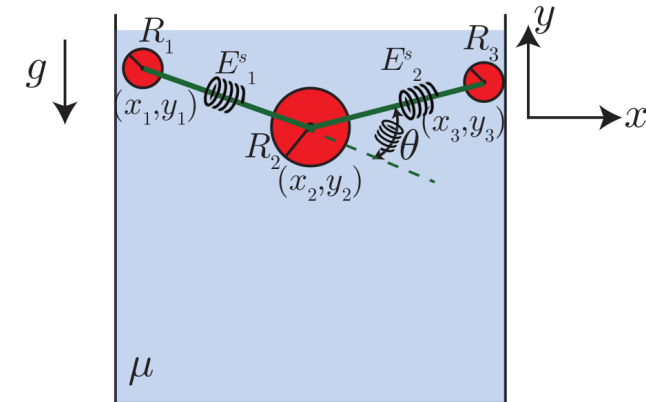
Alternative notation: $\boxed{\mathbf{M}} \ddot{\mathbf{q}} + \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} + \mathbf{C} \dot{\mathbf{q}} - \mathbf{W} = 0$

Mass matrix:

$$\mathbf{M} = \begin{bmatrix} m_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & m_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & m_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & m_2 & 0 & 0 \\ 0 & 0 & 0 & 0 & m_3 & 0 \\ 0 & 0 & 0 & 0 & 0 & m_3 \end{bmatrix},$$

where

$$m_1 = \frac{4}{3} \pi R_1^3 \rho_{\text{metal}}, m_2 = \frac{4}{3} \pi R_2^3 \rho_{\text{metal}}, m_3 = \frac{4}{3} \pi R_3^3 \rho_{\text{metal}},$$





Continuous Equations of Motion

$$\mathbf{M} \ddot{\mathbf{q}} + \boxed{\frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}}} + \mathbf{C} \dot{\mathbf{q}} - \mathbf{W} = \mathbf{0}$$

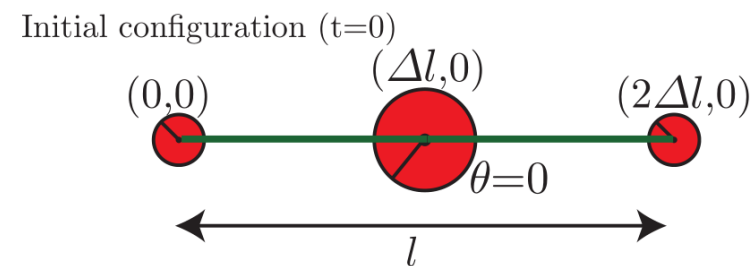
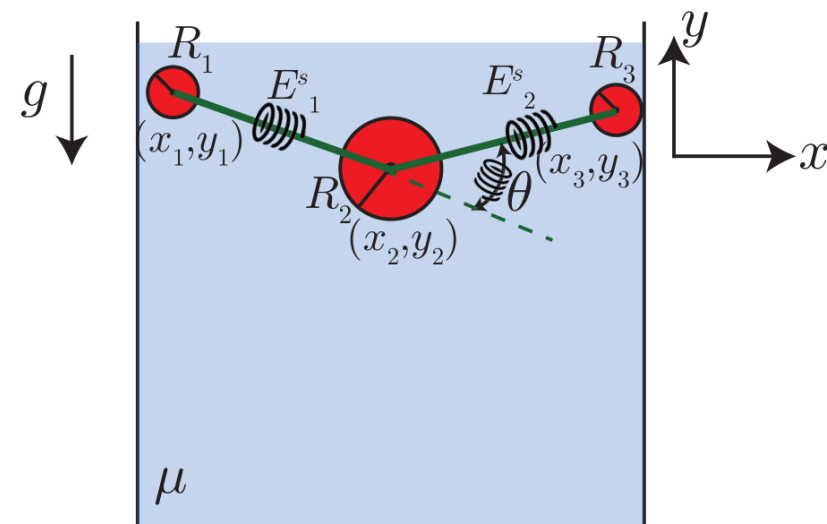
$$E^{\text{elastic}} = E_1^s + E_2^s + E^b,$$

where superscript s refers to *stretching*, b corresponds to *bending*, and

$$E_1^s = \frac{1}{2} EA \left(1 - \frac{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}{\Delta l} \right)^2 \Delta l,$$

$$E_2^s = \frac{1}{2} EA \left(1 - \frac{\sqrt{(x_3 - x_2)^2 + (y_3 - y_2)^2}}{\Delta l} \right)^2 \Delta l,$$

$$E^b = \frac{1}{2} \frac{EI}{\Delta l} \left(2 \tan \left(\frac{\theta}{2} \right) \right)^2.$$



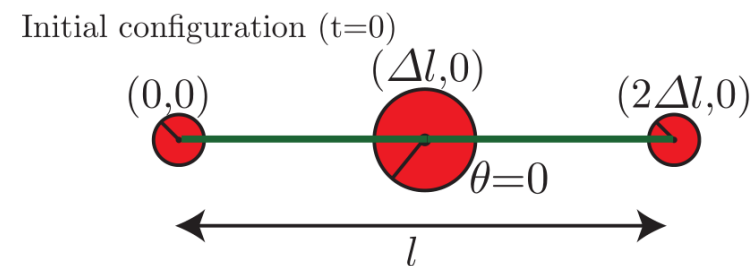
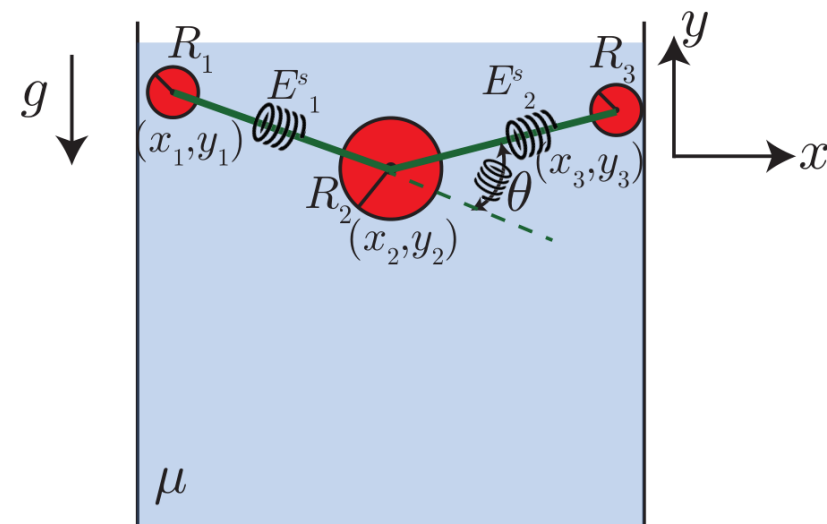


Continuous Equations of Motion

$$\mathbf{M} \ddot{\mathbf{q}} + \boxed{\frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}}} + \mathbf{C} \dot{\mathbf{q}} - \mathbf{W} = \mathbf{0}$$

$$\frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} = \frac{\partial E_1^s}{\partial \mathbf{q}} + \frac{\partial E_2^s}{\partial \mathbf{q}} + \frac{\partial E^b}{\partial \mathbf{q}} \rightarrow \text{Vector of size 6.}$$

Codes that compute this gradient are provided to you.





Continuous Equations of Motion

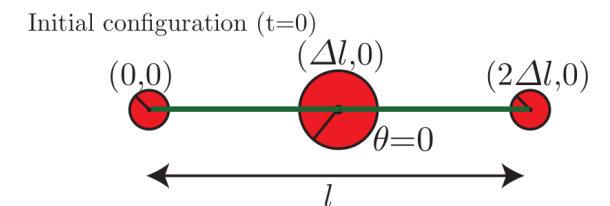
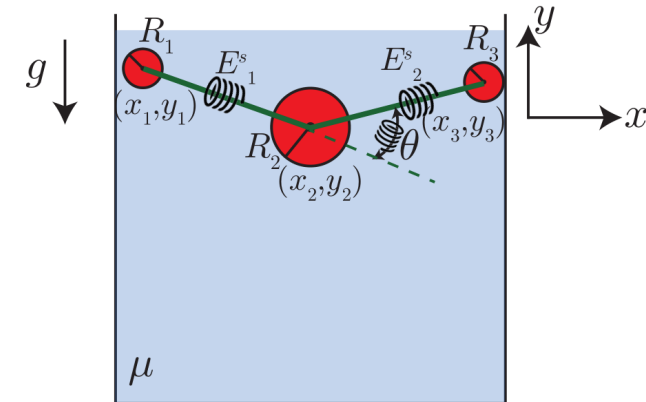
$$\mathbf{M} \ddot{\mathbf{q}} + \frac{\partial E^{\text{elastic}}}{\partial \dot{\mathbf{q}}} + \boxed{\mathbf{C}} \dot{\mathbf{q}} - \mathbf{W} = 0$$

Damping matrix:

$$\mathbf{C} = \begin{bmatrix} C_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & C_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & C_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & C_2 & 0 & 0 \\ 0 & 0 & 0 & 0 & C_3 & 0 \\ 0 & 0 & 0 & 0 & 0 & C_3 \end{bmatrix},$$

where

$$C_1 = 6\pi\mu R_1, C_2 = 6\pi\mu R_2, C_3 = 6\pi\mu R_3,$$





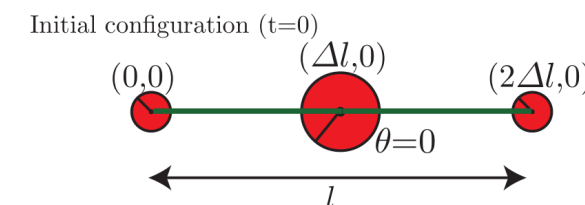
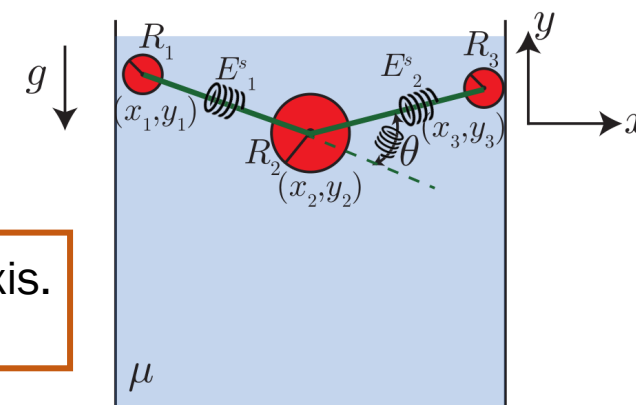
Continuous Equations of Motion

$$\mathbf{M} \ddot{\mathbf{q}} + \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} + \mathbf{C} \dot{\mathbf{q}} - \boxed{\mathbf{W}} = \mathbf{0}$$

Weight vector:

$$\mathbf{W} = \begin{bmatrix} 0 \\ W_1 \\ 0 \\ W_2 \\ 0 \\ W_3 \end{bmatrix},$$

Weight only acts along the y -axis.
Weight along the x -axis is 0.



where

$$W_1 = -\frac{4}{3}\pi R_1^3(\rho_{\text{metal}} - \rho_{\text{fluid}})g, \quad W_2 = -\frac{4}{3}\pi R_2^3(\rho_{\text{metal}} - \rho_{\text{fluid}})g, \quad W_3 = -\frac{4}{3}\pi R_3^3(\rho_{\text{metal}} - \rho_{\text{fluid}})g$$

Warning: be consistent with your sign convention.

Advice on Programming Implementation

- Elastic (conservative) forces

$$\frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} = \frac{\partial E_1^s}{\partial \mathbf{q}} + \frac{\partial E_2^s}{\partial \mathbf{q}} + \frac{\partial E^b}{\partial \mathbf{q}} \rightarrow \text{Vector of size 6.}$$

E_1^s only depends on x_1, y_1, x_2, y_2 . Therefore,

$$\frac{\partial E_1^s}{\partial \mathbf{q}} = \begin{bmatrix} \times \\ \times \\ \times \\ \times \\ 0 \\ 0 \end{bmatrix}$$

Non-zero vector of size 4.

Code that computes this non-zero portion is provided with the Course Notes.

Advice on Programming Implementation

- Elastic (conservative) forces

$$\frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} = \frac{\partial E_1^s}{\partial \mathbf{q}} + \frac{\partial E_2^s}{\partial \mathbf{q}} + \frac{\partial E^b}{\partial \mathbf{q}} \rightarrow \text{Vector of size 6.}$$

$$\frac{\partial E_s^1}{\partial \mathbf{q}} = \begin{bmatrix} \times \\ \times \\ \times \\ \times \\ 0 \\ 0 \end{bmatrix} \quad \frac{\partial E_s^2}{\partial \mathbf{q}} = \begin{bmatrix} 0 \\ 0 \\ \times \\ \times \\ \times \\ \times \end{bmatrix} \quad \frac{\partial E^b}{\partial \mathbf{q}} = \begin{bmatrix} \times \\ \times \\ \times \\ \times \\ \times \\ \times \end{bmatrix}$$

Advice on Programming Implementation

- Jacobian associated with elastic (conservative) forces
 - Also called the Hessian of elastic energy

$$\boxed{\frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q} \partial \mathbf{q}}} = \frac{\partial E_1^s}{\partial \mathbf{q} \partial \mathbf{q}} + \frac{\partial E_2^s}{\partial \mathbf{q} \partial \mathbf{q}} + \frac{\partial E^b}{\partial \mathbf{q} \partial \mathbf{q}} \rightarrow \text{Matrix of size } 6 \times 6.$$

Alternative notation: $\frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q}_i \partial \mathbf{q}_j}$ with $i = 1, 2, 3$ and $j = 1, 2, 3$.

$$\frac{\partial^2 E_s^1}{\partial \mathbf{q} \partial \mathbf{q}} = \begin{bmatrix} \times & \times & \times & \times & 0 & 0 \\ \times & \times & \times & \times & 0 & 0 \\ \times & \times & \times & \times & 0 & 0 \\ \times & \times & \times & \times & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \frac{\partial^2 E_s^2}{\partial \mathbf{q} \partial \mathbf{q}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times & \times \end{bmatrix} \quad \frac{\partial^2 E^b}{\partial \mathbf{q} \partial \mathbf{q}} = \begin{bmatrix} \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \end{bmatrix}$$



Advice on Programming Implementation

- Take advantage of the “block” nature of the forces and Jacobian

```
J = zeros(6,6);  
  
a = [ 1 2 3 4;  
      5 6 7 8;  
      9 1 2 3;  
      4 5 6 7 ];  
  
J(3:6, 3:6) = J(3:6, 3:6) + a;
```

```
J =  
  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 1 2 3 4  
0 0 5 6 7 8  
0 0 9 1 2 3  
0 0 4 5 6 7
```



Let us implement it



Module 12

General Case: Elastic Beam Falling in Viscous Fluid

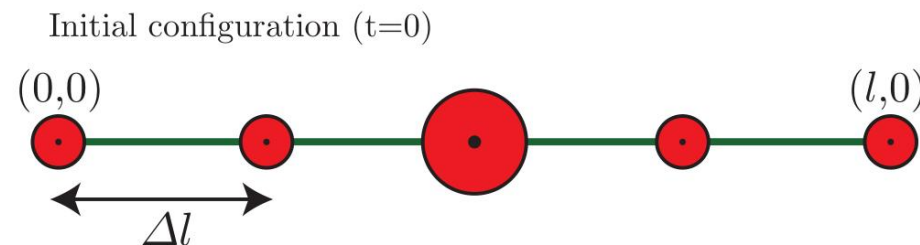
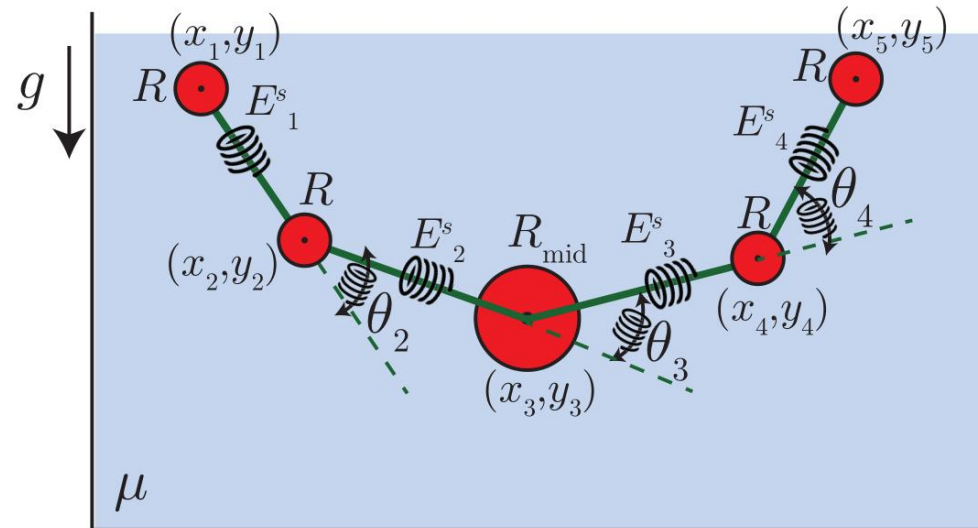
- Equations of Motion
- Advice on Programming Implementation

Goal



- Setup a problem with an elastic beam falling in viscous flow
- Formulate equations of motion and understand its implementation in software

- Read “Chapter 4: Conservative force and potential energy” of Course Notes



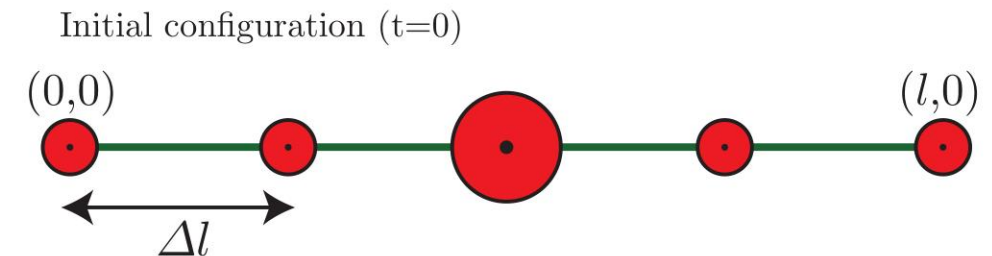
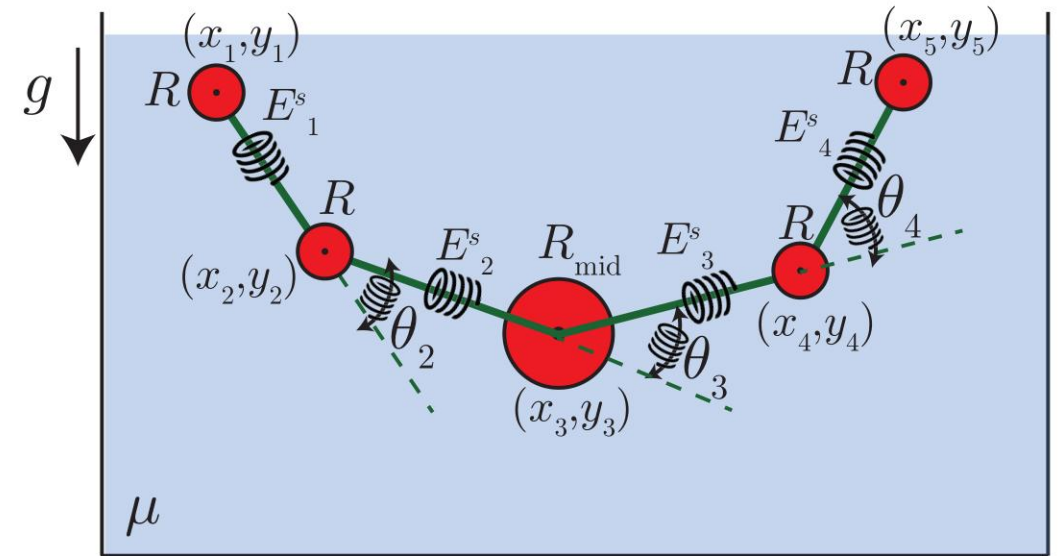


Problem Setup

N spheres (point masses) are connected with $(N-1)$ linear springs and $(N-2)$ torsional springs.

- Radii of the spheres (R_i) are known
- Viscosity of the fluid (μ) is known
- Initial configuration (Δl) is known
- Densities of the spheres (ρ_{metal}) and fluid (ρ_{fluid}) are known

Problem: simulate the shape of the system as a function of time



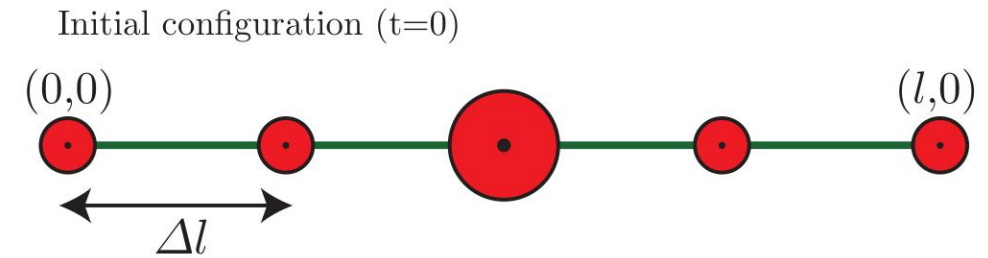
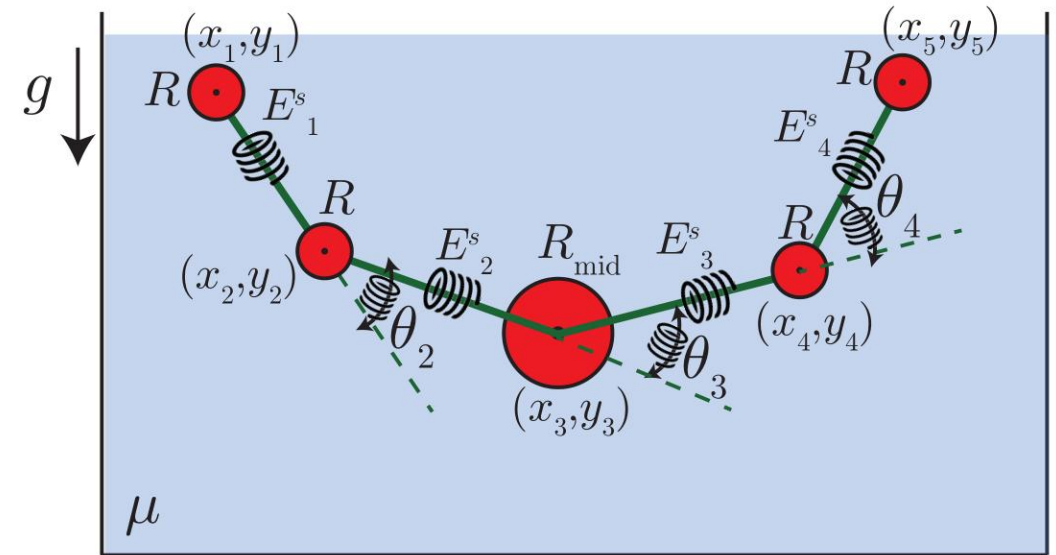


Degrees of Freedom (DOF)

DOF Vector with size $2N$:

$$\mathbf{q} = \begin{bmatrix} x_1 \\ y_1 \\ x_2 \\ y_2 \\ \dots \\ \dots \\ x_k \\ y_k \\ \dots \\ \dots \\ x_N \\ y_N \end{bmatrix}$$

k -th node corresponds to DOF $\# (2k - 1), 2k$





Continuous Equations of Motion

“Matrix” notation makes it easier to relate to programming implementation:

$$m_i \ddot{q}_i + \frac{\partial E^{\text{elastic}}}{\partial q_i} + c_i \dot{q}_i - w_i = 0$$



$$\mathbf{M} \ddot{\mathbf{q}} + \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} + \mathbf{C} \dot{\mathbf{q}} - \mathbf{W} = \mathbf{0}$$

Diagonal Mass Matrix
($2N \times 2N$)

Diagonal Damping Matrix
($2N \times 2N$)

Weight Vector
($2N \times 1$)

These quantities do not vary with time!



Discrete Equations of Motion (EOM)

$$\mathbf{M} \ddot{\mathbf{q}} + \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} + \mathbf{C} \dot{\mathbf{q}} - \mathbf{W} = 0$$



$$\mathbf{M} \frac{1}{\Delta t} \left(\frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t} - \dot{\mathbf{q}}(t_k) \right) + \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} + \mathbf{C} \left(\frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t} \right) - \mathbf{W} = 0$$

Known from previous time step

Unknown: compute by solving EOM

Don't forget to update velocities as well:

$$\dot{\mathbf{q}}(t_{k+1}) = \frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t}$$



Main Function in Software

$$[\mathbf{q}(t_{k+1}), \dot{\mathbf{q}}(t_{k+1})] = \text{SimulationLoop}([\mathbf{q}(t_k), \dot{\mathbf{q}}(t_k)])$$

Solve $\mathbf{x} \equiv \mathbf{q}(t_{k+1})$ using Newton's method
while $\text{err} > \epsilon$

$$\Delta \mathbf{x} = \mathbf{J}(\mathbf{x}) \backslash \mathbf{f}(\mathbf{x})$$

$$\mathbf{x} = \mathbf{x} - \Delta \mathbf{x}$$

$$\text{err} = \text{sum}(\text{abs}(\mathbf{f}(\mathbf{x})))$$

end of while

$$\dot{\mathbf{q}}(t_{k+1}) = \frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t}$$

We have to construct two functions:

- (1) $\mathbf{f}(\mathbf{q}(t_{k+1}))$ that returns a vector with size $2N \times 1$
- (2) $\mathbf{J}(\mathbf{q}(t_{k+1}))$ that returns a matrix with size $2N \times 2N$



$f(\mathbf{q}(t_{k+1}))$: “Force” Function

$$\mathbf{f} \equiv \mathbf{M} \frac{1}{\Delta t} \left(\frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t} - \dot{\mathbf{q}}(t_k) \right) + \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} + \mathbf{C} \left(\frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t} \right) - \mathbf{W}$$

Inertial force
Easy to compute

Damping force
Easy to compute

Weight
Easy to compute

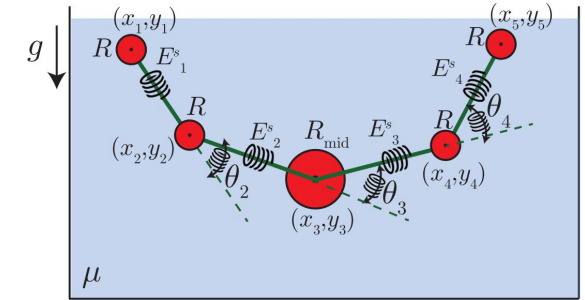
Energy Gradient: Vector of size $2N \times 1$
Exciting part



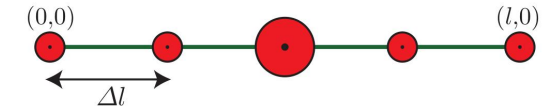
$f(q(t_{k+1}))$: “Force” Function

$$\mathbf{f}_e \equiv \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} \rightarrow (2N \times 1) \text{ sized vector}$$

“Big gradient vector” f_e



Initial configuration (t=0)

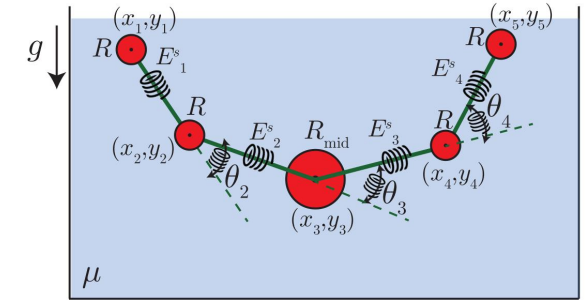




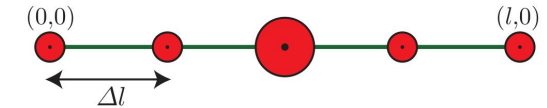
$f(q(t_{k+1}))$: “Force” Function

$$\mathbf{f}_e \equiv \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} \rightarrow (2N \times 1) \text{ sized vector}$$

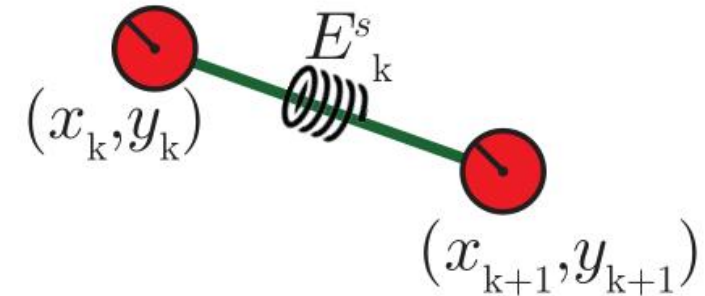
“Big gradient vector” $\mathbf{f}_e$



Initial configuration (t=0)



- Linear spring between k and $(k+1)$ -th nodes only affects DOF # $(2k-1)$, $2k$, $(2k+1)$, $(2k+2)$.
 - This corresponds to node # $k, k+1$
- “Small gradient vector” (size 4×1) $\mathbf{f}_k$ is computed by gradEs function in the Appendix
- $\mathbf{f}_e(2k-1:2k+2) = \mathbf{f}_e(2k-1:2k+2) + \mathbf{f}_k$
- Repeat for $k=1, \dots, N-1$

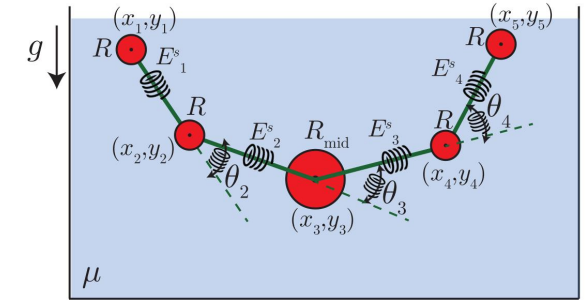




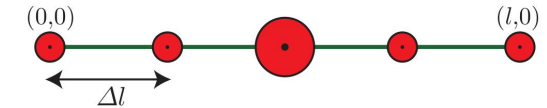
$f(q(t_{k+1}))$: “Force” Function

$$\mathbf{f}_e \equiv \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} \rightarrow (2N \times 1) \text{ sized vector}$$

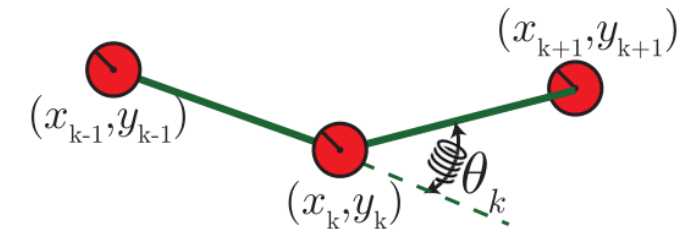
“Big gradient vector” $\mathbf{f}_e$



Initial configuration (t=0)



- Bending (torsional) spring at the k -th node only affects DOF # $(2k-3), (2k-2), (2k-1), 2k, (2k+1), (2k+2)$.
 - This corresponds to node # $k-1, k, k+1$
- “Small gradient vector” (size 6×1) $\mathbf{f}_k$ is computed by gradEb function in the Appendix
- $\mathbf{f}_e(2k-3:2k+2) = \mathbf{f}_e(2k-3:2k+2) + \mathbf{f}_k$
- Repeat for $k=2, \dots, N-1$





$J(\mathbf{q}(t_{k+1}))$: Jacobian Function

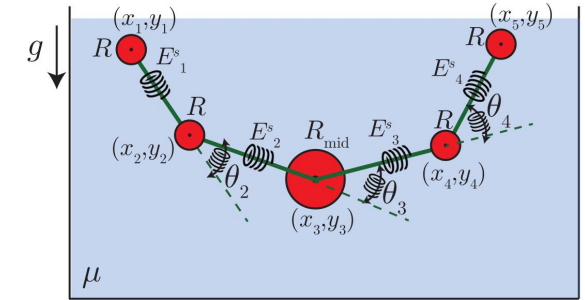
$$\mathbf{f} \equiv \mathbf{M} \frac{1}{\Delta t} \left(\frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t} - \dot{\mathbf{q}}(t_k) \right) + \frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}} + \mathbf{C} \left(\frac{\mathbf{q}(t_{k+1}) - \mathbf{q}(t_k)}{\Delta t} \right) - \mathbf{W}$$

$$\mathbf{J} \equiv \mathbf{J}_{ij} \equiv \frac{\partial \mathbf{f}}{\partial \mathbf{q}} \equiv \frac{\partial \mathbf{f}_i}{\partial \mathbf{q}_j}$$

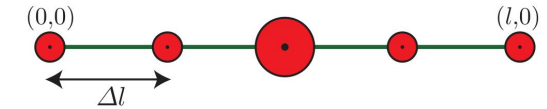
$$\mathbf{J} = \frac{\mathbf{M}}{(\Delta t)^2} + \boxed{\frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q} \partial \mathbf{q}}} + \frac{\mathbf{C}}{\Delta t}$$

$$\mathbf{J}_e \equiv \frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q} \partial \mathbf{q}} \equiv \frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q}_i \partial \mathbf{q}_j}$$

- Second derivative of elastic energy w.r.t. i -th and j -th DOFs
- $2N \times 2N$ sized matrix



Initial configuration ($t=0$)

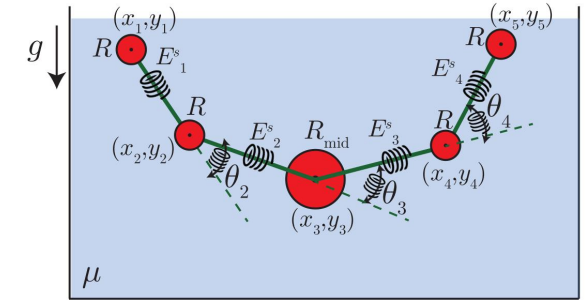




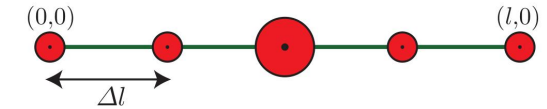
$J(q(t_{k+1}))$: Jacobian Function

$$\mathbf{J}_e \equiv \frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q}_i \partial \mathbf{q}_j} \rightarrow 2N \times 2N \text{ sized matrix}$$

"Big Hessian matrix" J_e



Initial configuration (t=0)

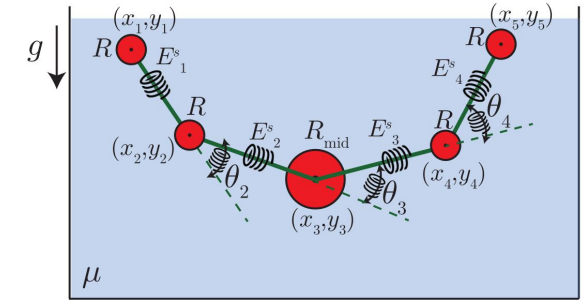




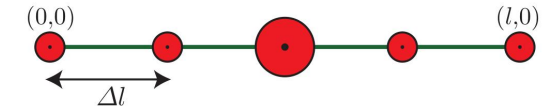
$J(q(t_{k+1}))$: Jacobian Function

$$\mathbf{J}_e \equiv \frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q}_i \partial \mathbf{q}_j} \rightarrow 2N \times 2N \text{ sized matrix}$$

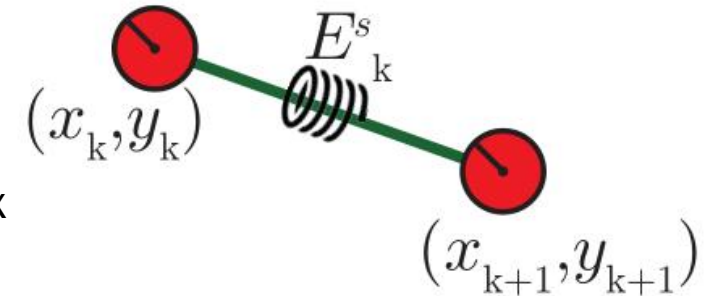
“Big Hessian matrix” J_e



Initial configuration (t=0)



- Linear spring between k and $(k+1)$ -th nodes only affects DOF # $(2k-1)$, $2k$, $(2k+1)$, $(2k+2)$.
 - This corresponds to node # k , $k+1$
- “Small Hessian vector” (size 4×4) J_k is computed by hessEs function in the Appendix
- $J_e(2k-1:2k+2, 2k-1:2k+2) = J_e(2k-1:2k+2, 2k-1:2k+2) + J_k$
- Repeat for $k=1, \dots, N-1$

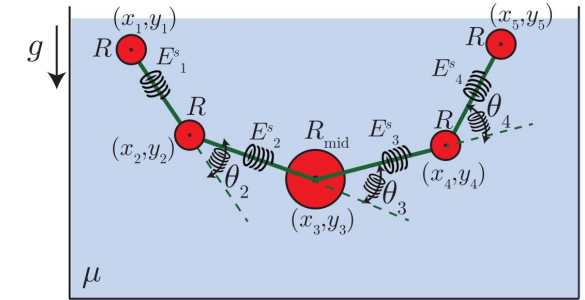




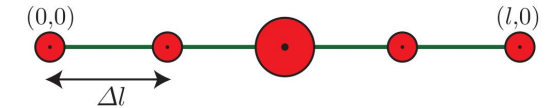
$J(q(t_{k+1}))$: Jacobian Function

$$\mathbf{J}_e \equiv \frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q}_i \partial \mathbf{q}_j} \rightarrow 2N \times 2N \text{ sized matrix}$$

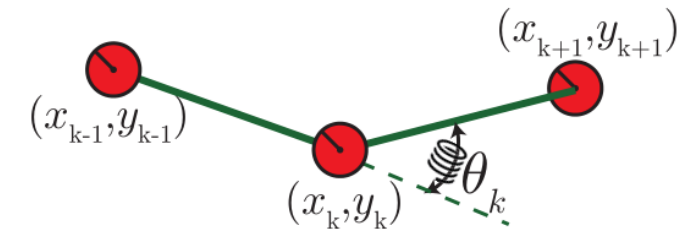
“Big Hessian matrix” J_e



Initial configuration (t=0)



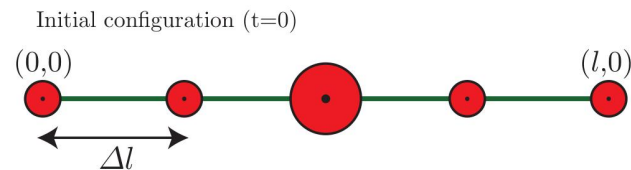
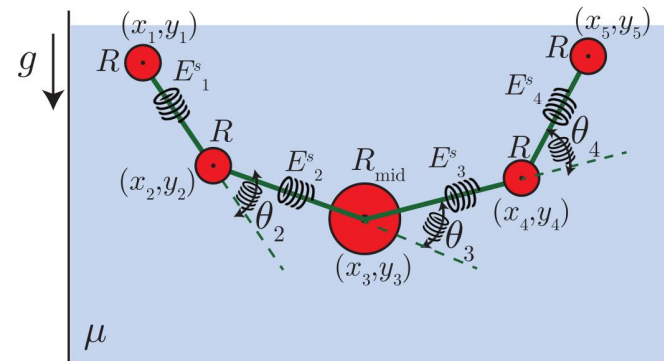
- Bending (torsional) spring at the k -th only affects DOF # $(2k-3), (2k-2), (2k-1), 2k, (2k+1), (2k+2)$.
 - This corresponds to node # $k-1, k, k+1$
- “Small Hessian matrix” (size 6×6) J_k is computed by hessEb function in the Appendix
- $J_e(2k-3:2k+2) = J_e(2k-3:2k+2) + J_k$
- Repeat for $k=2, \dots, N-1$



Assignments



- Complete **Assignment 2** in **Chapter 4**: *Generalized case of elastic beam falling in viscous flow*





Let us implement it
